SXE: Batch-Boosted, Holdout-Pruned, Sparsely Re-Weighted Histogram Trees

A tree ensemble between the random forest and the gradient-boosted machine

Alastair Rushworth

August 29, 2026

Prepared from the `sxe` repository at commit `eb97f58`; every measured figure traces to a script or artefact named in the text.

Abstract

SXE is a supervised tree-ensemble method for tabular data that occupies the space between two familiar constructions. A random forest grows many independent trees against the original target and averages them; a gradient-boosted decision tree (GBDT) grows one tree at a time against a running residual. SXE grows a small *batch* of independent, randomized trees at each boosting step, averages the batch, and applies that average as a single shrunken correction to the residual. The batch size is therefore a continuous dial between the two classical methods: a batch of one recovers stochastic gradient boosting, and a single batch spanning the whole budget recovers a random forest.

Two further devices sit on top of that generator. First, because each tree is grown on a row subsample, the complement of that subsample is a private holdout, and SXE uses it for reduced-error pruning at a strength drawn at random *per tree*, so the ensemble deliberately spans a spectrum of bias–variance trade-offs rather than committing to one. Second, after generation the whole trajectory is re-fitted by a single non-negative lasso over batch-average prediction columns on a held-out fold; most batches receive weight exactly zero and are physically deleted from the model. At the shipped defaults this discards 77–92% of the trees that were grown.

We give a complete description of the method and its implementation, and a head-to-head evaluation against a *tuned* LightGBM on nine datasets with a common size-accounting rule applied to both sides. SXE wins on accuracy on four datasets and loses on five; it carries the smaller model on five of nine, by up to 11.8×. Predictions are bit-identical across thread counts by construction. The clearest result in the study is negative: SXE's shipped defaults give up a median 9.5% error reduction to tuning where LightGBM's give up 3.3%, so the competitive column is a tuned SXE. We analyse why the accuracy wins and losses fall where they do — the method is a variance-reduction device, and it wins where variance dominates and loses where bias does — and we are explicit about what the evaluation does not establish.

Contents

1 Introduction **3**
1.1 Two constructions and the gap between them 3
1.2 What SXE is 3
1.3 What the design is for 4
1.4 Contributions 4

2 Notation and setting **4**

3 The method **5**
3.1 Global quantile binning . . . 5
3.2 Exclusive feature bundling . . . 6
3.3 Tree growth . . . 6
3.4 Holdout reduced-error pruning . . . 7
3.5 Batch-boosted dictionary generation . . . 8
3.6 The sparse non-negative weight fit . . . 9
3.7 Prediction, classification and calibration . . . 10
3.8 Determinism . . . 10
3.9 Serialization and model size . . . 11
3.10 Complexity . . . 12

4 Implementation **12**
4.1 Layout . . . 12
4.2 Build . . . 13
4.3 Threading . . . 13
4.4 scikit-learn conformance . . . 13

5 Experimental setup **13**
5.1 Datasets . . . 13
5.2 Protocol . . . 14
5.3 Provenance . . . 14

6 Results **15**
6.1 Head-to-head, both tuned . . . 15
6.2 Paired analysis . . . 15
6.3 Model size . . . 16
6.4 What tuning buys each library . . . 16
6.5 What the missing features cost . . . 17
6.6 Ablations . . . 17

7 Discussion **18**
7.1 Why the wins and losses fall where they do . . . 18
7.2 The lasso's real role . . . 19
7.3 An observation on fold reuse . . . 19

8 Limitations and threats to validity **19**

9 Related work **20**

10 Future work **21**
10.1 Unbuilt levers already paid for . . . 21
10.2 Then, in expected order of value . . . 21
10.3 Changes landed since the benchmark run . . . 22

11 Reproduction **22**

12 Conclusion **23**

A Parameter reference **24**

B Tuning guidance **25**

C Diagnostics exposed by a fitted model **26**

1 Introduction

1.1 Two constructions and the gap between them

Two ensemble constructions dominate supervised learning on tabular data, and they differ in how their members are related to one another.

The **random forest** [4] grows M trees that are mutually independent given the data. Each sees a bootstrap sample of the rows and a random subset of features at each split, each is fitted to the *original* target, and the prediction is their unweighted average. Independence is the point: averaging M estimators with correlation ρ and variance σ^2 gives variance $\rho\sigma^2 + (1-\rho)\sigma^2/M$, so decorrelating the members buys variance reduction that more members alone cannot. The construction is embarrassingly parallel, and its bias is the bias of a single deep tree — which it does not attempt to reduce.

The **gradient-boosted decision tree** [8] grows trees in strict sequence. Tree m is fitted to the negative gradient of the loss at the current ensemble prediction — for squared error, the residual — and is added with a shrinkage factor ν. Each member exists only as a correction to its predecessors. This attacks bias directly and is why GBDTs dominate tabular benchmarks [12], but it is inherently sequential: tree $m{+}1$ cannot begin until tree m has updated the residual. Modern implementations [14, 6] recover parallelism *within* a tree — across features and histogram bins — rather than across trees.

The gap between them is a design space, not a dichotomy. A boosting step need not be one tree. If a step consists of B trees grown independently against the *same* residual and then averaged, the step is a miniature random forest, the residual update happens once per step rather than once per tree, and the B trees within a step can be grown concurrently. Nothing about gradient boosting requires $B = 1$; the choice is historical.

1.2 What SXE is

SXE ("Sparse eXtra Ensembles") takes that observation and builds three further ideas on it.

1. **Batch boosting.** A boosting step grows B randomized, subsampled trees in parallel against the current residual and applies their average as one shrunken correction. B is a parameter (`batch_size`), and it interpolates continuously between stochastic gradient boosting ($B = 1$) and a random forest (one batch).

2. **Per-tree holdout pruning at randomized strength.** Each tree is grown on a random fraction of the rows, so the complement is a holdout that tree has never seen. SXE routes it through the grown tree and runs classical reduced-error pruning [16] bottom-up, collapsing any subtree that does not beat its own collapsed leaf on holdout squared error by a per-leaf penalty λ. Crucially λ is drawn *per tree* from a log-uniform distribution, so the ensemble contains members ranging from essentially unpruned to nearly stumps. No single bias–variance trade-off is chosen at fit time; the final stage arbitrates.

3. **One-shot sparse non-negative re-weighting.** After generation, SXE discards the uniform weights that boosting assigned and re-fits the trajectory with a non-negative lasso on a held-out fold, using one column per *batch* (the batch's average prediction) rather than one per tree. Most coefficients come out exactly zero, and the corresponding batches are deleted from the model object. This is Friedman and Popescu's ISLE construction [9] — generate a dictionary of trees, then fit a sparse linear combination of them — with Breiman's non-negativity constraint from stacked regressions [3], applied at batch granularity.

The per-tree randomization (depth, minimum leaf size, feature fraction, subsample rate, split rule, pruning strength all drawn independently for every member) is in the spirit of extremely

randomized trees [11]: the method averages over hyperparameter draws instead of committing to a single setting. The histogram machinery — quantile pre-binning, parent-minus-child histogram subtraction, exclusive feature bundling, optional leaf-wise growth — follows LightGBM [14].

1.3 What the design is for

The method is not proposed as a general accuracy improvement over tuned GBDTs; Section 6 shows it is not one. It targets three properties that follow from its structure:

Model size.
The lasso deletes most of the generated ensemble, and the survivors were already pruned against honest holdout data. Where this lands well the model is several times smaller than a tuned LightGBM's at comparable accuracy.

Reproducibility.
Every tree's random number stream is seeded from its *index* in the generation order, never from which worker thread picked it up. The number of threads is therefore a pure performance knob: the same seed produces bit-identical predictions on 1 core or 10.

Parallel structure.
Trees within a batch are independent, so growth parallelises across trees, which sequential GBDT cannot do. Only the between-batch residual update is serial.

1.4 Contributions

1. A complete specification of the method (Section 3), including the exact split criterion, the pruning inequality and its degenerate case, the batch-boosting loop, and the weight fit.
2. An implementation account (Section 4) covering the hybrid histogram grower, the thread pool, the determinism argument, and the serialization format.
3. A head-to-head evaluation against a tuned LightGBM on nine datasets under one size-accounting rule, with paired per-seed analysis (Section 6).
4. A quantified negative result: the shipped defaults are substantially worse than the method (Section 6.4), with evidence about *which* defaults are wrong.
5. An analysis of where the method's accuracy wins and losses fall and why (Section 7), and an explicit account of what the study does not establish (Section 8).

2 Notation and setting

Training data are $(x_i, y_i)_{i=1}^n$ with $x_i \in \mathbb{R}^p$. Write $X \in \mathbb{R}^{n \times p}$ for the design matrix and $y \in \mathbb{R}^n$ for the target.

The loss is squared error throughout. Regression minimises mean squared error directly. Binary classification is handled as regression on $y \in \{0, 1\}$ — the Brier-score reduction — so leaf means are conditional-probability estimates and a squared-error split on 0/1 targets is equivalent to a Gini split up to constants. There is no logistic loss and there are no second-order (Newton) leaf values; Section 8 returns to what this costs. Multiclass is a one-vs-rest reduction over the binary core.

SXE partitions the training data once, before anything else happens. A fraction ρ_{val} (`validation_fraction`, default 0.18) is set aside as a validation fold $(X_{\text{val}}, y_{\text{val}})$ of size n_{val}; the remainder is the generation set on which all trees are grown. The validation fold is used for early stopping, for the weight fit, and for probability calibration.

The fitted model has the form

$$\hat{y}(x) \;=\; \beta_0 \;+\; \sum_{b \in \mathcal{S}} w_b \cdot \underbrace{\frac{1}{B} \sum_{t \in \text{batch } b} T_t(x)}_{\text{batch mean}}, \qquad w_b \geq 0, \tag{1}$$

where each T_t is a pruned regression tree, B is the batch size, and $\mathcal{S}$ is the sparse set of batches surviving the weight fit. Members of batches outside $\mathcal{S}$ are deleted from the model object rather than merely zero-weighted, which is what makes the size claim about the artefact and not about a hypothetical pruning.

3 The method

Figure-free, the pipeline is: bin once → (optionally bundle binary columns) → generate a dictionary of trees by batch boosting, each tree self-pruned → fit non-negative sparse weights on the validation fold → delete everything with zero weight → prune the bin table to the columns that survive.

3.1 Global quantile binning

Every feature is quantile-binned once, up front, and *every* downstream stage — split search, holdout routing, pruning, member prediction, the weight-fit design matrix — operates on that single `uint8` matrix. There is no intermediate float matrix.

`max_bins` counts the bins available to finite values and is at most 255, following scikit-learn's `HistGradientBoosting*` vocabulary; the 256th bin is reserved for missing values and is not part of the budget. Edges are estimated from a subsample of at most 50,000 rows: the code takes `max_bins` -1 interior points of linspace$(0, 1, \cdot)$ as quantile levels, deduplicates the resulting edges (so a feature with few distinct values gets few bins), and maps values with `searchsorted(edges, x, side='right')`. The layout handed to the C++ core is column-major, since the split search sweeps a feature at a time.

Binning is not merely a speed device here. A uniform random threshold drawn on a binned feature is *exactly* a random bin cut-point, so the extremely-randomized split rule of Section 3.3 composes with binning without approximation — unlike a random threshold on raw values, which binning would have to round.

Missing values. `NaN` gets its own bin. At the default `max_bins`= 255 the finite values occupy bins 0..254 and missing occupies 255, which needs no support in the tree code at all: bin 255 routes like any other. Missing therefore travels with the largest values by default, and a split at the top finite bin — which the exact-cut scan finds whenever it pays — separates it. Infinity is still rejected at `fit` and `predict`: `NaN` means missing, but infinity is a number the binning cannot place. An all-missing column gets no edges at all and every row lands in bin 255.

This is weaker than LightGBM's learned default direction, which can send missing either way at every node, and the repository is explicit that it is a convenience feature rather than an accuracy feature. The evidence is mixed by design: on a synthetic problem where missingness *is* the signal the scheme reaches 0.97 AUC, which no imputation can reach; but against median imputation at 10% and 30% missingness on four datasets it wins four of eight comparisons and loses four, with Covertype the clearest loss — punching holes in binary indicator columns puts `NaN` above the positive class, where the median puts it with the negatives. The point is that the library stops refusing the data, not that it handles it better than the user would.

3.2 Exclusive feature bundling

`BinaryBundler` implements LightGBM-style exclusive feature bundling [14] restricted to strictly binary columns, and is off by default (`bundle_binary_features`). Columns whose values lie in $\{0, 1\}$ are ordered by descending nonzero count and greedily assigned to the first bundle whose used-row mask they do not intersect; conflict tolerance is zero and a bundle holds at most 250 members. A bundle of size ≥ 2 becomes one small-integer column: 0 if no member is active, else the 1-indexed position of the active member. Singletons pass through unchanged.

The encoding is lossless up to ordinality. A threshold split on a bundle selects a *contiguous group* of the bundled categories rather than isolating one, which is weaker than a one-vs-rest split, but trees compose splits and the measured accuracy effect was nil. On Covertype this compresses 54 features to 12 — bundles of 5 wilderness and 39 soil indicators — and cuts histogram work by roughly 4.5×.

Exclusivity is verified on the training data and must hold at predict time; a prediction row that activates two members of one bundle would otherwise be silently encoded as the first of them, indistinguishable from a row where only that member is set. The current code raises, naming the offending row and columns.

3.3 Tree growth

Trees are grown by the C++ core on the binned matrix. For a node containing m rows with target sum S, a candidate split of feature f at bin threshold t (routing "bin $\leq t$" left) is scored by the variance-reduction proxy

$$G(f, t) \;=\; \frac{S_L^2}{n_L} \;+\; \frac{S_R^2}{n_R} \;-\; \frac{S^2}{m}, \tag{2}$$

where (n_L, S_L) and (n_R, S_R) are the counts and target sums of the two children. This is the standard reduction in sum of squares, written so that only prefix sums of the node's histogram are needed. A candidate is invalid if either child would fall below `min_samples_leaf`. A node stops splitting when it reaches the tree's depth cap, when $m < 2\,$`min_samples_leaf`, or when its target is numerically pure.

Candidate generation. At each node, K distinct candidate features are drawn without replacement by a partial Fisher–Yates shuffle, with $K = \mathrm{round}(\phi p)$ and $\phi \sim U($`feat_lo`$,$ `feat_hi`$)$ drawn *once per tree*. For each candidate feature, cut-points are evaluated in one of two modes:

Exact (`splitter="best"`, the default).
A prefix scan over the feature's occupied bin range returns the maximiser of (2).

Extremely randomized (`splitter="random"`).
`n_thresholds` cut-points are drawn uniformly on the occupied bin range and only those are scored.

With a retained histogram both cost $O(\text{bin span})$, so the exact scan is essentially free; the choice is not about speed but about the trade between split quality and member diversity, and it is genuinely dataset-dependent — the tuned configurations of Section 6.4 chose random cuts on the high-dimensional and one-hot datasets and exact cuts elsewhere.

Hybrid depth-wise growth with histogram subtraction. Nodes with at least `hybrid_min` rows (default 600) retain a full all-feature histogram of $p \times 256$ (count, sum) pairs. Candidates are then evaluated from prefix sums with no pass over the data at all. After the node's index range is partitioned in place, the *smaller* child's histogram is built from data in $O(|\text{small}| \cdot p)$

and the larger child's is obtained by an in-place parent-minus-child subtraction, reusing the parent's buffer. At most depth + 1 buffers are live per thread and they are pooled.

Children below `hybrid_min` fall back to a second grower that builds a single 256-bin histogram per candidate feature with touched-bin clearing, and so pays no $p \times 256$ clearing cost on small nodes. The threshold trades one cost against the other; 600 was found to work across all three of the datasets it was tuned on and has not needed a per-dataset value since.

Observation 1 (`hybrid_min` is a pure performance knob)**.** Both growers select splits identically but draw from one shared per-tree random stream, so the order in which they recurse determines which node a given random draw lands on. Both therefore recurse *smaller child first*. Without that, changing `hybrid_min` — ostensibly a speed setting — would silently change the fitted model. The repository tests this invariance rather than asserting it.

Leaf-wise growth. Setting `max_leaf_nodes` switches to best-first growth in LightGBM's style: a max-heap keyed on gain holds pending leaves, the highest-gain leaf is split, each child's best split is evaluated and pushed if it clears the depth, size and gain floors, and growth stops at the leaf budget. This is not a corner case — the random search of Section 6.4 selected it on four of nine datasets.

A pending leaf carries only its best split, not the histogram that found it; the histogram is rebuilt when the leaf is popped and released immediately, so at most one is live per workspace. That gives up parent-minus-child subtraction for the children of a popped node, costing roughly one extra histogram fill per node, and it is nonetheless *faster*: the alternative — one retained `Hist` per pending leaf, at $p \times 256 \times 12$ bytes each — made the workspace scale with the leaf budget rather than with depth, and its per-buffer reset `memset` dominated the fill it was saving. On 4000×500 data at `batch_size`= 16 the retained-histogram scheme reached 906 MB peak RSS in 0.90 s against 254 MB in 0.62 s for the rebuild scheme, and 304 MB for depth-wise growth.

Exported node ids satisfy parent < children in both modes, which is what makes the bottom-up passes below valid without a separate traversal order.

Per-tree randomization. Every tree independently draws:

Quantity	Distribution
`max_depth`	uniform over `max_depth_choices` ($0 \Rightarrow$ capped at 64)
`min_samples_leaf`	log-uniform on `min_samples_leaf_range`
feature fraction ϕ	uniform on `max_features_range`
row subsample rate	uniform on `subsample_range`, sampled without replacement
pruning strength λ	log-uniform on `prune_strength_range`

No two members of a batch share hyperparameters. This is the ExtraTrees idea carried up a level: rather than choosing one configuration, the ensemble averages over the draw.

3.4 Holdout reduced-error pruning

Each tree is grown on a subsample of rate drawn in $[0.4, 0.75]$ by default, so 25–60% of the rows are *not* used to grow it. These are that tree's private holdout, and they are free — the out-of-bag idea [2] applied to pruning rather than to scoring.

After growth, the holdout rows are routed through the tree once, accumulating $(n_h, \sum y, \sum y^2)$ at *every* node on each row's root-to-leaf path. A single bottom-up pass — descending node id, valid because ids are pre-order — then computes, for each node, the holdout sum of squared errors that collapsing that node to a leaf would incur:

$$\mathrm{SSE}_{\mathrm{leaf}} \;=\; \sum_h y^2 \;-\; 2\mu \sum_h y \;+\; n_h \mu^2, \tag{3}$$

where μ is the node's *training* mean — the value the collapsed leaf would actually predict. The node is collapsed if and only if

$$\mathrm{SSE}_{\mathrm{leaf}} \;\le\; \mathrm{SSE}_{\mathrm{subtree}} \;+\; \lambda\,\hat{\sigma}_h^2\,(L_{\mathrm{subtree}} - 1), \tag{4}$$

where $\mathrm{SSE}_{\mathrm{subtree}}$ is the sum of the (possibly already collapsed) children's values, L_{subtree} is the subtree's post-collapse leaf count, and $\hat{\sigma}_h^2$ is the holdout variance at the root, which makes λ scale-free.

The interpretation is direct. At $\lambda = 0$ the rule removes exactly those splits that do not help on data the tree has never seen — dead wood. As λ grows, each additional leaf must pay for itself at a rate of $\lambda\hat{\sigma}_h^2$, giving a nested sequence of increasingly aggressive subtrees: a holdout analogue of weakest-link cost-complexity pruning [5], but with the complexity penalty evaluated against honest data rather than resubstitution error.

Each tree emits exactly one member, pruned at its own $\lambda \sim \log\text{-}U(\lambda_{\mathrm{lo}}, \lambda_{\mathrm{hi}})$. Pruning diversity therefore lives *across* trees, and the weight fit of Section 3.6 decides how much of each strength the model wants. Collapsed nodes are exported as leaves carrying their training means.

Observation 2 (The small-holdout degeneracy)**.** If the subsample rate approaches 1, the holdout is nearly empty and many subtrees are visited by no holdout row at all. For such a subtree both sides of (4) are $0 \le 0 + \lambda\hat{\sigma}_h^2(L-1)$, which is satisfied trivially, and the tree collapses — in the worst case to a stump. SXE guards this by emitting the tree unpruned whenever the holdout has fewer than $\max(50, n/50)$ rows. The fix was worth $0.4504 \to 0.4437$ RMSE on California housing in the original measurement, and it is what makes mild-subsampling configurations viable at all.

3.5 Batch-boosted dictionary generation

The generation loop runs entirely in the C++ core. Let r denote the residual on the generation set and F_{val} the running prediction on the validation fold.

Algorithm 1 Batch boosting (`Ensemble::boost`)

$r \leftarrow y$; $\quad F_{\mathrm{val}} \leftarrow 0$; $\quad$ best $\leftarrow \infty$; $\quad$ bad $\leftarrow 0$
for $b = 1, \ldots,$ `max_iter` **do**
  in parallel for $t = 1, \ldots, B$:
    seed tree (b, t) from its index; draw its hyperparameters
    draw a row subsample; grow a tree on (X, r)
    route the complement; prune by (4) at its own λ
    evaluate T_t on the generation rows and on X_{val}
  $r \leftarrow r - \dfrac{\nu}{B}\displaystyle\sum_{t=1}^{B} T_t(X)$ ▷ one shrunken correction per batch
  $F_{\mathrm{val}} \leftarrow F_{\mathrm{val}} + \dfrac{\nu}{B}\displaystyle\sum_{t=1}^{B} T_t(X_{\mathrm{val}})$
  mse $\leftarrow \|y_{\mathrm{val}} - F_{\mathrm{val}}\|^2 / n_{\mathrm{val}}$
  if mse $<$ best $- \tau$ **then** best $\leftarrow$ mse; bad $\leftarrow 0$
  else if `++bad` $\ge$ `n_iter_no_change` **then break**
  end if
end for

The early-stopping tolerance is $\tau = \texttt{tol} \cdot \mathrm{Var}(y_{\mathrm{val}})$, so it is expressed in units of the target's variance rather than in absolute error. The residual update is the only serial dependency in the loop; the B tree constructions inside a batch touch no shared mutable state.

Two properties of the loop are worth naming. First, trees fit residuals, and their holdout pruning is evaluated against the same residual target — which is the correct objective at that stage, since the tree's job is to explain what remains. Second, the batch average $\frac{1}{B}\sum_t T_t$ is itself a small random forest, so each boosting step is a shrunken correction built from a variance-reduced estimate rather than from a single high-variance tree. This is the sense in which B dials between the two classical methods:

`batch_size`	Recovers	Character
1	stochastic gradient boosting	one tree per residual update
10–16 (default range)	SXE	a mini-forest per residual update
→ whole budget	random forest	one update, everything fitted to y

Empirically $B \in [4, 16]$ preserved almost all of boosting's accuracy while exposing the parallel width.

3.6 The sparse non-negative weight fit

Generation produces up to `max_iter` $\times B$ trees with uniform implied weights ν/B. SXE discards those weights and re-fits.

Let $P \in \mathbb{R}^{n_{\text{val}} \times M}$ hold every member's prediction on the validation fold. Columns are averaged within batches, $P_b = \frac{1}{B}\sum_{t \in b} P_{\cdot t}$, giving one column per batch; this averaging happens inside the C++ core so the design matrix is never materialised at member granularity. The weights solve a positive elastic net with intercept — a pure lasso at the default `l1_ratio`= 1:

$$\min_{w \geq 0,\, \beta_0} \; \frac{1}{2n_{\text{val}}} \left\| y_{\text{val}} - \beta_0 - P_b w \right\|_2^2 \; + \; \alpha \|w\|_1. \tag{5}$$

The regularisation strength α is chosen by 3-fold cross-validation over a path of 20 values, with Gram matrices precomputed — which took the fit from roughly 20 s to 1–5 s at thousands of columns and to 0.02–0.3 s at hundreds. Batches whose weight is zero are deleted; each surviving member receives weight w_b/B, and β_0 becomes the model intercept. If the cross-validated solution collapses entirely to the intercept, a tiny-α positive fit is substituted so the model remains an ensemble rather than a constant.

Four design choices deserve justification.

Why a lasso at all. By the stagewise–lasso correspondence [7, 13], forward stagewise fitting with infinitesimal steps traces the lasso path. The boosting trajectory is exactly such a stagewise fit, so a lasso over the generated columns is a convex stand-in for the greedy procedure that produced them — it can express the same solutions, and it can express sparser ones the greedy fit passed through and never returned to. This is ISLE [9].

Why non-negative. Breiman's stacked regressions [3] found that constraining combination weights to be non-negative yields sparser and markedly more stable solutions than unconstrained least squares. It also prevents the pathology where two batches acquire large opposing weights that cancel: every retained batch must contribute in the direction it was fitted. Note that the weights are *not* additionally constrained to sum to one — the fit is free to scale the ensemble, and the intercept absorbs whatever shared bias remains.

Why on held-out data. Selection is deliberately on the validation fold. Fitted in-sample, the deepest and least-pruned members would absorb everything and the pruning diversity of Section 3.4 would be wasted.

Why batch-average columns. Cost. Per-member columns made the fit explode to $\approx 7\,\mathrm{s}$ at thousands of columns; batch columns made it 0.02–0.3 s with no measured accuracy change, and the fit's cost becomes independent of the tree count. The price is granularity: a batch is retained or dropped as a unit, and the model cannot express "keep three of these ten trees".

Observation 3 (The lasso compresses; it does not correct)**.** A diagnostic comparing the fitted weights against the raw uniform generation weights ν/B found test-set-equal accuracy with roughly $10\times$ compression. The weight fit's role is therefore *trajectory pruning* — deciding which steps of the boosting path are redundant — rather than repairing the direction of the fit. Generation decides where each batch points; the lasso decides how much of it to keep, and whether to keep it at all.

The prototype's earlier multi-round class (`CppSXE` in `reference/`) retains a different schedule for the same fit — full cross-validation on the first and last rounds, single warm fixed-α fits at $0.7\times$ the previous α in between, and a narrowed final grid $\alpha_1 \cdot \mathrm{logspace}(0.5, -1, 10)$ — which the one-shot fit described here replaced.

Measured at the shipped defaults on two datasets (script in Section 11):

Dataset	Batches run	Trees grown	Trees kept	Kept
California housing	300	4,800	368	7.7%
Friedman1 + noise	155	2,480	560	22.6%

The California figure is not an accident of that dataset: the defaults budget 300 batches and early stopping never fires, so the generator runs its full course and the lasso then discards 92% of what it built.

3.7 Prediction, classification and calibration

Prediction routes binned rows through the retained trees and forms $\beta_0 + \sum_j w_j T_j(x)$, parallelised over rows.

Binary classification predicts the same raw score. Because the score is a weighted combination of 0/1 leaf means it is approximately a probability but not a calibrated one, so SXE fits a Platt sigmoid [15] on the validation fold: $p = \sigma(a\,s + b)$ by maximum likelihood against Platt's smoothed targets $(N_+ + 1)/(N_+ + 2)$ and $1/(N_- + 2)$, which keep the optimum finite when the fold happens to be separable. The smoothing is applied by giving every observation both labels with weights t and $1 - t$, so an ordinary logistic solver optimises the smoothed objective exactly. If the fold cannot identify a calibration — one class only, a constant score, or a fitted map that is non-increasing — the calibrator is dropped and the raw score is clipped to $[10^{-6}, 1 - 10^{-6}]$ instead, so a bad fit can never reorder the scores. Under the one-vs-rest reduction this decision is made for all classes together: a rare class whose validation fold contains no positives cannot be calibrated, and mapping the others while clipping that one would normalise a row across two different scales.

Multiclass is one-vs-rest: one complete ensemble per class, with per-class scores normalised to sum to one. This is a reduction, not a native objective, and it costs a full ensemble per class — on the 26-class Letter dataset, 26 ensembles.

3.8 Determinism

The random stream for the tree at batch b, slot t is seeded as

$$\mathrm{seed}_{b,t} \;=\; \texttt{splitmix64}\big(s \;+\; \texttt{0x9E3779B9} \cdot (bB + t + 1)\big), \tag{6}$$

a function of the tree's *index* only. Nothing about which worker thread executes the tree, or in what order the pool drains its queue, enters the stream. Combined with the smaller-child-first recursion of both growers, this gives:

- identical models across `n_jobs` settings;
- identical models across `hybrid_min` settings;
- identical models across repeated runs.

All three are covered by tests in the repository (`tests/test_determinism.py`), not merely asserted. The invariance is *within* a platform. Per-tree randomness is drawn through `std::uniform_int_distribution` and `std::uniform_real_distribution`, whose algorithms are implementation-defined by the C++ standard, so libstdc++, libc++ and MSVC produce different sequences from the same seeded `mt19937_64` and therefore grow different trees. Making seeds portable would require replacing the standard distributions with fixed implementations; the repository records this as an open decision.

3.9 Serialization and model size

Two distinct quantities must be kept apart.

An earlier version of this comparison set an SXE format estimate against *pickled* baselines, which overstated the difference by two to five times; that claim was withdrawn, and the rule below is what replaced it.

The portable accounting formula. For comparing tree structure, size is computed identically for both sides: 13 bytes per internal node (feature 4, threshold 1, left 4, right 4), 8 bytes per leaf (value), plus a full 255-edge float64 bin table per input feature. Trees are full binary, so internal nodes = leaves − trees. This is the column reported in Section 6, and it special-cases neither library — but it compares *structure* and nothing else, and it is a poor guide to what either library would ship. It charges LightGBM for a bin table its format does not need, and it charges both sides for edges that deduplicate away on low-cardinality data or are never indexed on wide data. The repository measures SXE's real bin table at 1.9 kB on Letter where the formula charges 32.6 kB, and at 67 kB on Madelon where the formula charges 1.02 MB.

A second, less flattering column. The benchmark script has since been extended to record a `real_bytes` column alongside the portable one: measured for SXE (`model_size_bytes_`, the bytes `serialize()` writes plus the quantile edges of the columns the trees actually index), and for LightGBM the smallest sensible encoding of what its trees need — a float64 threshold rather than a bin id, an explicit left child, and no bin table, at 20 bytes per internal node. That assumption is generous to LightGBM, whose real files also carry split gains and node counts. It is nonetheless *less* favourable to SXE than the portable formula: on Madelon LightGBM's real model is the smaller one, which the portable column hid behind a bin table neither library actually pays. The committed results predate this change, so Section 6 reports the portable column throughout and Section 8 carries the consequence.

The format SXE actually writes. `Ensemble::serialize` stores only what prediction reads. Three fields that a naive layout would store are recovered or discarded instead:

- *left child*: the exporter emits nodes in pre-order, pushing right then left, so a node's left child is always the next id. It is recomputed on load.
- *value on internal nodes*: prediction reads a value only at a leaf.
- *gain*: it existed for `feature_importances_`, which is now accumulated once at fit time into a length-p vector.

What remains is 9 bytes per internal node, 8 per leaf, and one bit per node recording which is which. The bin table is additionally pruned to the columns the retained trees actually index; on Madelon at defaults the model splits on 61 of 500 features and was carrying 421 kB of quantile edges for the other 439. Unused columns transform to bin 0, which is safe precisely because nothing reads them, and a one-vs-rest classifier shares a single table across its sub-models covering the union of the columns they use, rather than each sub-model holding a pruned copy of the same edges.

An earlier version of this format wrote 29 bytes per node — storing `gain` everywhere, `feat/thr/left/right` on leaves and `val` on internal nodes — so the advertised size was 2.9× smaller than the artefact. That has been corrected. Measured at the shipped defaults:

Dataset	`serialize()`	Portable formula	Ratio
California housing	472,757 B	565,618 B	0.84×
Friedman1 + noise	1,279,555 B	1,542,667 B	0.83×

SXE's real tree format is therefore *below* the portable formula the benchmark charges it. Its bin-table charge runs the other way, and by more, so the two errors do not cancel and neither sign can be claimed for the size column as a whole.

3.10 Complexity

Let n_s be a tree's subsample size, n_h its holdout size, d its depth, and M_b the number of batch columns.

Stage	Cost
Growth (with subtraction)	$O(n_s(p+d))$ per tree
In-place partitioning	$O(n_s d)$ per tree
Holdout routing	$O(n_h d)$ per tree
Pruning pass	$O(\text{nodes})$ per tree
Residual/validation update	$O((n+n_{\text{val}})d)$ per tree
Weight fit	$O(n_{\text{val}}M_b^2)$ Gram, then coordinate descent on $M_b \times M_b$

The $O(n_s p)$ term in growth is dominated by the smaller-child histogram fills, which sum to at most $n_s p/2$ per level because the larger child is always obtained by subtraction. Histogram memory is $O(\text{depth})$ per thread in depth-wise growth — at most depth + 1 pooled buffers live — and $O(1)$, a single live buffer, in leaf-wise growth; neither scales with the leaf budget or the tree count. Measured single-core grower throughput on Covertype was ≈ 27.8k leaves/s against LightGBM's ≈ 13.9k, so wall-clock differences between the libraries come from *how much structure each method builds*, not from per-leaf cost.

4 Implementation

4.1 Layout

Path	Role
`src/sxe/_core.cpp`	C++17 core: growers, pruning, boosting, prediction, serialization
`src/sxe/estimator.py`	`SXERegressor`, `SXEClassifier`
`src/sxe/binning.py`	`QuantileBinner`
`src/sxe/bundling.py`	`BinaryBundler`
`benchmarks/`	the head-to-head suite and its committed outputs
`reference/`	the original ctypes prototype, kept as a parity oracle

The division of labour is deliberate: everything that runs once per fit and is cheap (binning, bundling, the train/validation split, the lasso) stays in Python where scikit-learn's implementations are already good; everything that runs once per tree or once per row is in C++.

The prototype under `reference/` is not installed and is retained for two purposes: as the source of the v1/v2 ablation figures in Section 6, and as a parity oracle — the packaged core reproduces it bit-for-bit. It holds `sxe.cpp` (the original C API core with OpenMP, including the v2 multi-round `sxe_grow`), `cpp_sxe.py` (a ctypes wrapper exposing `CppSXE` for v2 and `CppSXEBoost` for v3), `sparse_extra.py` (the pure-Python v1 prototype built on scikit-learn's `ExtraTreeRegressor` with cost-complexity-path variants), the v1/v2 benchmark scripts, and `results_v3.csv`.

4.2 Build

The package builds with scikit-build-core and pybind11 and needs a C++17 compiler and CMake from source; at runtime it needs only NumPy ($\geq$ 1.22) and scikit-learn ($\geq$ 1.3) on Python $\geq$ 3.9, with `lightgbm` required only by the benchmark scripts. Continuous integration builds and tests wheels for five platforms across CPython 3.9–3.13.

Optimisation flags are deliberately portable: `-march=native` measured no reproducible speedup on these datasets, which is consistent with the memory-bound histogram loops of Section 3.10, so wheels give it up for nothing.

4.3 Threading

Growth and prediction use a hand-rolled `std::thread` pool rather than OpenMP, so there is no runtime to bundle — notably no `libomp` step for macOS users — and `n_jobs` is explicit. The pool holds threads $-$ 1 workers and uses the calling thread as the last one. Work is distributed by an atomic counter, so scheduling is dynamic; this is safe for determinism precisely because each work item's random stream is index-derived (Section 3.8). The first exception thrown by any worker is captured, the counter is poisoned so remaining workers stop promptly, and the exception is rethrown on the calling thread. The GIL is released for the whole generation loop.

`n_jobs=None` means *all cores*, matching other gradient-boosting libraries rather than scikit-learn's single-core default; negative values follow the joblib convention.

4.4 scikit-learn conformance

`SXERegressor` and `SXEClassifier` pass `sklearn.utils.estimator_checks.check_estimator` in full, so they compose with `Pipeline`, `GridSearchCV`, `cross_val_score` and `joblib` without adapters. Fitted models pickle and deep-copy — the trees travel through the serialization format of Section 3.9 — so they survive the round trip into joblib workers. Parameter names follow scikit-learn's `HistGradientBoosting*` vocabulary wherever the concepts match. The repository's test suite (105 tests at the time of writing) covers the API surface, determinism, persistence, multiclass, calibration, validation and output quality.

5 Experimental setup

5.1 Datasets

Nine datasets, chosen so that each stresses a different property — including several that stress properties SXE does not have.

Dataset	Shape	Task	What it tests
California housing	$20{,}640 \times 8$	regression	smooth low-dimensional signal, small n
Friedman1 + 20 noise	$25{,}000 \times 25$	regression	known signal buried in noise
Superconductivity	$21{,}263 \times 81$	regression	many correlated informative features
Covertype 2-vs-rest	$30{,}000 \times 54$	binary	the original reference point
Covertype 2-vs-rest, full	$581{,}012 \times 54$	binary	large n, where GBDT scaling shows
Adult	$48{,}842 \times 14$	binary	categoricals (cardinality 2–41), missing values
Bank marketing	$45{,}211 \times 16$	binary	categoricals (cardinality 2–12)
Letter recognition	$20{,}000 \times 16$	26-class	the cost of the one-vs-rest reduction
Madelon	$2{,}600 \times 500$	binary	$p \gg n$, mostly irrelevant features

Five seeds each, except Covertype-full and Letter at three. The seed drives the 80/20 train/test split (stratified for classification), the model, and — for Friedman1 — the data itself.

5.2 Protocol

Both libraries are tuned. A win over a misconfigured baseline establishes nothing, so both libraries are tuned per dataset by random search over a fixed space with the same number of draws (10–24, larger where a fit is cheap). Draw zero is always the library's own defaults, so a tuned row cannot be worse than its default row on the selection fold. The search runs once per dataset on seed 0 and the selected configuration is reused across seeds, so neither library tunes against a seed it is scored on. The hand-tuned configurations from the ablation history (Section 6) are entered as additional warm starts. Selected configurations are committed to `benchmarks/tuned_configs.json`.

Common preprocessing. Both libraries receive the same matrix: categorical columns ordinal-coded, numeric missing values median-imputed from the training split. This is necessary because SXE at the reference commit rejects non-finite input and has no categorical handling. A separate `LightGBM-native` row runs LightGBM on raw `NaN`s with declared categorical columns, which prices what the workaround costs.

One size rule, both sides. Size is the portable formula of Section 3.9 applied to both libraries' structure counts.

Timing. Wall-clock on one 10-core laptop; each library is reported at its own best thread count. On this machine LightGBM's best is one thread on seven of nine datasets and SXE's is all cores on all nine.

One asymmetry, in LightGBM's favour. LightGBM's early-stopping fold doubles as its selection fold. SXE early-stops on a fold carved from the training half and is therefore selected on data it has not seen.

5.3 Provenance

Every figure in Section 6 is recomputed directly from `benchmarks/results_suite.csv`, the committed raw output of `benchmarks/bench_suite.py`. Accuracy and structure are read at the lowest thread count so that repeated thread settings do not double-count; time is the mean at each library's best thread count.

Table 1: Both libraries tuned. Mean ± standard deviation across seeds. Bold marks the better mean.

		Accuracy		Size (MB)		Best fit (s)	
Dataset	Metric	LightGBM	SXE	LGBM	SXE	LGBM	SXE
California	rmse ↓	**0.4409** ± 0.0095	0.4480 ± 0.0097	2.845	**0.241**	2.22	**1.40**
Friedman1+noise	rmse ↓	1.0695 ± 0.0112	**1.0557** ± 0.0166	**0.526**	0.993	**1.50**	2.00
Superconduct	rmse ↓	**9.3313** ± 0.3171	9.6685 ± 0.2807	5.766	**1.253**	13.33	**9.63**
Covertype 30k	auc ↑	0.9547 ± 0.0031	**0.9590** ± 0.0026	**1.092**	3.545	**1.32**	8.32
Covertype full	auc ↑	**0.9977** ± 0.0001	0.9946 ± 0.0001	**10.508**	14.448	**50.31**	82.11
Adult	auc ↑	**0.9305** ± 0.0016	0.9240 ± 0.0020	0.489	**0.443**	1.72	**1.69**
Bank marketing	auc ↑	0.9338 ± 0.0017	**0.9354** ± 0.0021	0.202	**0.181**	**1.02**	2.24
Letter (26)	acc ↑	**0.9661** ± 0.0018	0.9629 ± 0.0003	**5.109**	13.564	67.61	**33.85**
Madelon	auc ↑	0.8873 ± 0.0089	**0.9031** ± 0.0075	1.364	**1.036**	**3.66**	5.96

6 Results

6.1 Head-to-head, both tuned

Accuracy: four to SXE, five to LightGBM. Size: five to SXE, four to LightGBM. Time: three to SXE, six to LightGBM. No axis is a systematic win.

The timing column deserves one note about what it is not. Reported this way — each library at its own best thread count — speed goes to SXE on three of nine. At the setting a user is actually likely to choose, all cores for both, SXE would win most of them, because LightGBM's within-tree parallelism costs it on data this size. That framing is available and is declined here: a library compared against itself at its worst configuration is the same error as a tuned model compared against an untuned baseline.

6.2 Paired analysis

Means and standard deviations across seeds understate what a paired comparison can say, because both libraries see the same split at each seed. Table 2 reports the per-seed difference.

Table 2: Paired per-seed differences, SXE minus LightGBM. t is the mean difference in units of its own standard deviation; positive t favours SXE for AUC and accuracy, negative t favours SXE for RMSE.

Dataset	Metric	Mean diff	sd	$\lvert t \rvert$	Verdict
Covertype full	auc	−0.00313	0.00005	57.8	LightGBM, decisive (0/3 seeds)
Adult	auc	−0.00645	0.00066	9.8	LightGBM, clear (0/5)
Covertype 30k	auc	+0.00436	0.00088	5.0	SXE, clear (5/5)
Superconduct	rmse	+0.33720	0.10956	3.1	LightGBM, clear (0/5)
Madelon	auc	+0.01574	0.00661	2.4	SXE, clear (5/5)
Letter	acc	−0.00317	0.00166	1.9	LightGBM, suggestive (0/3)
Bank marketing	auc	+0.00157	0.00090	1.7	SXE, suggestive (5/5)
California	rmse	+0.00712	0.00629	1.1	within noise (0/5)
Friedman1+noise	rmse	−0.01380	0.01265	1.1	within noise (4/5)

Two SXE wins (Covertype-30k, Madelon) and three LightGBM wins (Covertype-full, Adult, Superconductivity) are clear. Bank marketing and Letter are suggestive but not decisive. California and Friedman1 are within seed-to-seed spread, though the sign is consistent across seeds in each case. Note that the sign of the mean difference is consistent with the win count on every dataset, so no result is an artefact of one outlying seed.

6.3 Model size

Size is smaller for SXE on five of nine, and where it wins it wins large: 11.8× on California, 4.6× on Superconductivity. But the mechanism deserves stating plainly, because it is not "SXE builds smaller models". Neither random search penalises size. Both columns therefore report where an accuracy-driven search happened to land, and LightGBM's search drifts to very large models — 135k leaves on California, 267k on Superconductivity, 497k on Covertype-full — whenever that buys it anything at all. Where SXE loses on size it loses on structure it genuinely spends: 164k leaves on Covertype-30k, and 650k on Letter, where the one-vs-rest reduction pays for 26 separate ensembles.

One caution on reading these numbers by proxy: leaf count is not a format-free stand-in for size. The two libraries put different amounts of information in a node and a leaf, and SXE additionally carries a bin table that LightGBM does not, so a leaf-count ratio and a byte ratio can point in different directions.

The five–four split is also specific to the portable accounting. Under the real-format column of Section 3.9 the Madelon verdict reverses, because the portable formula charges both libraries for a 500-feature bin table that SXE prunes to 61 columns and LightGBM never needs at all. Read the size column as a structure comparison, which is what it is.

6.4 What tuning buys each library

The gap between a library's tuned and default rows is the measurable form of "does this need tuning". Table 3 reports the error reduction from default to tuned, where error is RMSE for regression and $1 - \text{metric}$ for classification.

Table 3: Error reduction from each library's defaults to its tuned configuration. Larger means the defaults were further from what the library can do.

Dataset	SXE	LightGBM	Ratio
California	4.4%	1.0%	4.3×
Friedman1+noise	1.9%	3.3%	0.6×
Superconduct	11.8%	2.4%	5.0×
Covertype 30k	50.4%	8.2%	6.2×
Covertype full	82.8%	61.4%	1.3×
Adult	4.5%	2.7%	1.7×
Bank marketing	2.6%	0.9%	2.9×
Letter	36.8%	6.7%	5.5×
Madelon	9.5%	3.6%	2.6×
median	**9.5%**	**3.3%**	**2.9×**

SXE gains more from tuning on eight of nine datasets. This is the clearest result in the study and it is negative. It is a statement about the shipped defaults rather than about the method — the tuned column of Table 2 is competitive, the default one is not — but a user's first fit is what they actually get, so the distinction is cold comfort.

The winning configurations say specifically what is wrong. Not one of the nine used the default `max_depth_choices` $= (4, 5, 6, 7, 8)$: five chose $(8, 10, 12)$ or $(10, 12, 14)$ and four chose leaf-wise growth with 63–127 leaves. `max_iter` selected the top of its search range on seven of nine and `batch_size` on six. Three of the knobs that matter most therefore ran into the ceiling of the search space, which means the defaults are not merely mistuned but *systematically build too small a model*, and the true optimum lies beyond what was searched.

Those defaults have since been revised in exactly that direction (Section 10.3), recovering a median 58% of the per-dataset tuning gain on six datasets re-checked, at roughly 4× the fit time and 2–3.5× the model size. The table above is measured at the previous settings and is

left as the diagnosis that motivated the change; what the revision is worth across the whole suite is a question for the next full run.

6.5 What the missing features cost

Dataset	LightGBM (shared matrix)	LightGBM-native	Difference
Adult	0.9305 auc	0.9304 auc	−0.0001
Bank marketing	0.9338 auc	0.9351 auc	+0.0013

RandomForest context. For scale, the v1-era measurements against scikit-learn's `RandomForest` (single seed, prototype, not re-run): California 0.511 RMSE in 33.8 s, Friedman1 1.443 RMSE in 116 s, Covertype 0.9486 AUC in 11.5 s. Every version of SXE from v1 onward beats those on both accuracy and time by wide margins, which is the floor the method clears rather than a competitive comparison. The v1-era model sizes were pickled scikit-learn objects and are not comparable to the size column above.

Native categorical handling and native missing-value handling buy LightGBM almost nothing on these two datasets. SXE's gap on Adult (−0.0065 AUC) is therefore a *model* gap, not an encoding gap — a useful negative result, since it says that adding native categorical support would not close it. What the missing features cost is user effort: the preprocessing must happen, and today the user writes it.

6.6 Ablations

The following deltas were measured on the original prototype, under Linux/libstdc++, single-seed. They are retained as the *relative* evidence for each design decision — the direction and rough magnitude of each change — and will not reproduce exactly against Section 6, which is a multi-seed re-measurement on macOS/libc++.

Change	Effect
Multi-round boosting vs. a single-round dictionary	The single-round dictionary lands at random-forest level; the residual rounds supply essentially all accuracy beyond bagging. Boosted multi-round beat RF everywhere (Friedman 1.088 vs. 1.443 RMSE) at 20–40× smaller models; LightGBM matched it only on the noisy task.
Holdout pruning replacing a cost-complexity approximation	Covertype AUC 0.9480 → 0.9528, alongside 5–10× faster growth from the move to C++, and a 6× cut in the weight fit from precomputing the Gram matrix. Consistent with pruning decisions made against genuinely held-out data.
Histogram subtraction	≈ 1.6× on growth.
Exclusive feature bundling	A further ≈ 2.4× on Covertype at unchanged accuracy. Profiling at this stage showed the grower already producing ≈ 2× more leaves/s than LightGBM, so the remaining wall-clock gap was structure volume and pipeline overhead rather than per-leaf cost. Covertype became a clean three-axis win; Friedman was accuracy-tied but slow, because the dictionary was a fixed size where LightGBM early-stopped; California plateaued near 0.47 RMSE.
Batch boosting + batch-group lasso (v2 → v3)	Friedman flipped to a win on all three axes — a 0.9-second grower and a 0.12 MB model. Per-member lasso columns cost ≈ 7 s at thousands of columns; batch columns made the fit 0.02–0.3 s with no accuracy change. The shallow-tree defaults broke Covertype outright (AUC 0.92), recovered only by depth 10–14 with random cuts: that dataset genuinely needs deep, wide-candidate trees, and it was the first clear sign of the defaults problem of Section 6.4.
Small-holdout guard	Observation 2. California 0.4504 → 0.4437 RMSE.
Leaf-wise growth	Did not break the California accuracy plateau (63 leaves ≈ depth-wise best, 31 leaves worse) — evidence that the residual gap there is not about tree shape.

7 Discussion

7.1 Why the wins and losses fall where they do

Every device in SXE is a variance-reduction device. Averaging B independent trees per step reduces the variance of each correction. Randomizing depth, leaf size, feature fraction and subsample rate decorrelates the members. Reduced-error pruning against a private holdout removes structure that does not generalise. The non-negative lasso deletes whole steps of the trajectory that do not pay on held-out data. There is no corresponding device for bias: the loss is squared error, leaf values are training means, and there is no Newton step.

The results follow that structure closely. SXE wins where variance dominates the error:

- **Madelon** ($p \gg n$, 2,600 rows, 500 features, mostly irrelevant) — the regime where a single greedy learner overfits the irrelevant features and averaging over feature subsets pays most. SXE wins by 2.4 sd on 5/5 seeds, with a model 1.3× smaller.
- **Covertype-30k** — a subsampled problem where the same argument applies at moderate scale. 5.0 sd, 5/5 seeds.

- **Friedman1 with 20 pure-noise features** — signal buried in noise; SXE leads on 4/5 seeds, though within spread.

  And it loses where bias dominates:

- **Covertype-full** (465k training rows) is the sharpest case. With that much data there is enough signal to fit the bias away, and a sequential leaf-wise learner does that better than an averaged forest of randomized trees. The gap is small in absolute terms (0.0031 AUC) but 57.8 sd — utterly consistent.
- **Superconductivity** (81 correlated informative features) — randomizing the feature subset hurts when most features carry signal, because the diversity it buys costs split quality that the data would have supported.
- **California housing** (smooth, low-dimensional, strong signal) sits at a plateau that neither leaf-wise growth nor deeper trees moved, which points at leaf *values* rather than tree *shape* as the binding constraint — consistent with squared-error training means being the remaining approximation.

This is a coherent story rather than a set of scattered outcomes, and it is actionable: the levers most likely to close the remaining gaps are all on the bias side (Section 10).

7.2 The lasso's real role

It is tempting to read the weight fit as a corrective stage that improves on the boosting trajectory. The diagnostic says otherwise: fitted weights and raw uniform generation weights give equal test accuracy, at roughly 10× compression. The lasso is a *compression* stage. That is exactly what the method needs it to be — the size claim is its entire justification — but it also means the accuracy of SXE is the accuracy of the batch-boosted generator, and improvements should be sought there.

7.3 An observation on fold reuse

The single validation fold does four jobs: early stopping, the lasso weight fit, the cross-validation of α inside that fit, and Platt calibration. The α selection is properly nested within the fold, so that is sound. But the batches retained by the lasso are selected on the same fold that early stopping already optimised against, so the retained set — and therefore the model size, which is the headline claim — is chosen on data the generator has already been steered toward. The test sets are untouched, so Section 6's accuracy figures are not affected; the concern is specifically that the compression ratio is measured under mild optimism. A second fold, or a cross-fitted weight fit, would settle it.

8 Limitations and threats to validity

Stated at the strength the evidence supports.

The defaults are not the method. Section 6.4 is the dominant caveat. Everything in the SXE column of Table 2 is a tuned SXE, and at the settings those results were measured under, a user's first fit gave up a median 9.5% error reduction. The defaults have since been revised and recover a median 58% of that on the datasets re-checked (Section 10.3), but the revision has not been measured across the whole suite, so the honest description of the package remains "competitive after a search, and better out of the box than these tables show".

Size is an outcome, not a controlled variable. Both searches optimise accuracy alone, so a single tuned point per library cannot answer the question the use case actually poses — *what accuracy is available if the model has to fit in N megabytes*. The search has since been instrumented to record every draw's (loss, size) pair, which answers it at no extra cost, but the results reported here predate that and report only where an accuracy-driven search happened to land.

The size column compares structure, not artefacts. The portable accounting is applied identically to both sides, which is the right call for comparing tree structure, but it is a byte count of neither library. It charges LightGBM for a bin table its format does not need, and it charges both sides for quantile edges that deduplicate away or are never indexed — SXE's real bin table is 1.9 kB on Letter against the formula's 32.6 kB. The net effect is not uniformly in SXE's favour: its own tree format is measurably *below* the formula (Section 3.9), but its bin-table charge is far above what it ships, and under the real-format accounting the Madelon size verdict reverses. The correct reading is that structure is compared fairly and shipped size is not yet compared at all.

Nine datasets, one machine. Timings are wall-clock on a 10-core laptop and will not transfer. Three seeds on the two most expensive datasets is thin for a variance estimate.

Seeds are not portable across platforms. Bit-identical reproducibility holds across runs, thread counts and `hybrid_min` on one platform, and is tested. It does not hold across C++ standard libraries (Section 3.8).

Method limits. Squared error throughout; no logistic loss; no second-order leaf values. Multiclass is a one-vs-rest reduction costing one ensemble per class — on Letter, 26 ensembles, 650k leaves and 2.7× LightGBM's model for slightly worse accuracy. Features are capped at 255 finite bins plus the missing bin. There is no general categorical handling beyond binary bundling, and no `sample_weight`.

Preprocessing was done for the benchmark. Categoricals were ordinal-coded and missing values imputed before either library saw them. Section 6 prices what that costs in accuracy — almost nothing on this suite — but not what it costs in user effort, which is real. SXE now takes `NaN` directly (Section 3.1), so of that burden only the categorical encoding remains; the benchmark predates the change and imputed for both libraries anyway, which keeps the comparison model-against-model.

Concurrency. Concurrent `fit` calls on a single estimator instance are unsupported, as they are for scikit-learn estimators generally.

9 Related work

SXE is a recombination of established components rather than a new primitive, and its lineage is worth stating precisely.

The **dictionary-then-sparse-combination** structure is Friedman and Popescu's Importance Sampled Learning Ensembles [9]: generate a large family of trees by some randomized procedure, then fit a regularised linear combination of them. RuleFit [10] is the same idea at the granularity of individual rules (root-to-leaf paths) rather than whole trees. SXE sits at a *coarser* granularity than either — one column per batch — for cost reasons, and Section 10 notes the finer end as unexplored.

The **non-negativity constraint** on combination weights is Breiman's stacked regressions [3], motivated there by exactly the stability and sparsity properties SXE relies on.

The **lasso-as-stagewise** justification is the LARS correspondence [7] and its forward-stagewise analysis [13]: the boosting path and the lasso path coincide in the infinitesimal-step limit, which is what licenses fitting a convex problem in place of the greedy one.

The **pruning** is classical reduced-error pruning [16] against honest holdout data, with a complexity penalty in the style of cost-complexity pruning [5] but evaluated out of sample. The use of the out-of-subsample complement is the out-of-bag device [2] applied to pruning rather than to error estimation.

The **per-tree randomization** is in the spirit of extremely randomized trees [11], extended from the split threshold to the tree's whole hyperparameter vector.

The **histogram machinery** — pre-binning, parent-minus-child histogram subtraction, exclusive feature bundling, leaf-wise growth — is LightGBM's [14], as is much of the parameter vocabulary via scikit-learn's `HistGradientBoosting*`.

What is not, to our knowledge, standard is the specific combination: batch boosting as a continuous interpolation between forests and boosting; *randomizing the pruning strength across members* so the ensemble spans a bias–variance spectrum that a later convex fit arbitrates; and applying the sparse non-negative fit at batch granularity so that it prunes the boosting *trajectory* rather than individual members.

10 Future work

10.1 Unbuilt levers already paid for

Three refinements target leaf *values* rather than tree structure, which Section 7 identifies as where the remaining gap most plausibly lives. All three are cheap given what the pipeline already computes.

Honest leaf values.
The holdout is already routed through every tree and its per-node $(n_h, \sum y, \sum y^2)$ are already accumulated for the pruning decision — but exported leaves carry *training* means. Re-estimating leaf values from the holdout, as honest trees do in the causal-forest literature [17], is nearly free given what is in hand.

Hierarchical shrinkage.
Shrink each node's prediction toward its parent in proportion to sample counts, then delete nodes whose post-shrinkage contribution vanishes [1]. Reported to often beat hard pruning on accuracy, and it composes with the existing bottom-up pass rather than replacing it.

Leaf-level re-weighting.
The weight fit operates on batch-average columns, coarser than per-tree. The fine end is a ridge or lasso over the leaf-membership matrix — RuleFit territory [10] — which can express corrections that per-batch weights cannot.

10.2 Then, in expected order of value

1. **Fix the defaults.** Section 6.4 names the direction: deeper trees, more batches, larger batches. Because three knobs hit the ceiling of the search space, the first step is a wider search, not a re-pick within the current one. This is the highest-value work in the list by a distance, because it is the difference between the method and the product.
2. **Newton (second-order) leaf values and a learning-rate schedule**, both aimed at the bias-side gap.

3. **Logistic loss and a native multiclass objective**, which would retire the one-vs-rest reduction and the 26× cost on Letter.
4. **A size-budgeted evaluation**, so the size claim answers the question the use case poses.
5. **A GPU port**, for which the batch structure is unusually well-suited: within a batch, all trees × all nodes at a depth × all candidates reduce to gather and histogram kernels with no sort or argmax over raw values, and the weight fit is one small GEMM plus coordinate descent.

10.3 Changes landed since the benchmark run

Four changes have landed since the results of Section 6 were measured. None of them alters the method; two of them change what a user gets, and the committed benchmark output predates all four.

Missing-value support, as described in Section 3.1. This removes the preprocessing burden priced in Section 6 — though, as that section shows, the preprocessing was costing accuracy almost nothing, so what it removes is user effort.

A defaults revision. `learning_rate` $0.15 \to 0.07$, `max_iter` $120 \to 300$, `batch_size` $10 \to 16$, `n_iter_no_change` $8 \to 15$, and `max_depth_choices` $(4, 5, 6, 7, 8) \to (6, 8, 10, 12)$ — one configuration derived from the pattern in Section 6.4 rather than tuned per dataset, moving all three of the identified knobs the way the tuned configurations pointed. On six datasets re-checked it improves every one and recovers a median 58% of the per-dataset tuning gain. It is not free: fits take roughly 4× longer and default models are 2–3.5× larger, which trades directly against the size claim. That is the right trade while default accuracy is the larger gap, and the five parameters can be set back. A full re-run of the suite is what would settle it; per the project's own rule, a default change is a product decision measured across the whole benchmark, not on one dataset.

The dual size accounting of Section 3.9, together with a record of every tuning draw's (loss, size) pair in `tuning_draws.csv`, so the size-budget question of Section 8 becomes answerable on the next full run.

Two benchmark-protocol changes. Covertype-full is now timed at all cores only, as Letter already was — a single-threaded fit at the new defaults on 465k rows runs to ten minutes, and the thread sweep would dominate the suite for a comparison nobody would make at that size — and the SXE search cap rises from 120 to 200 batches so sampled draws are not held below the default trajectory length.

A leaf-wise memory fix (Section 3.3) and a publishing path: a `v*` tag now pushes wheels to PyPI by trusted publishing, though the package is not yet installable from PyPI.

11 Reproduction

```
pip install -e ".[bench]"

python benchmarks/bench_suite.py                    # full suite, ~1.5 h
python benchmarks/bench_suite.py -d adult letter    # a subset
python benchmarks/bench_suite.py --seeds 0          # single seed, quick
python benchmarks/bench_suite.py --retune           # redo both random searches

python -m pytest tests/                             # 105 tests
```

Raw output lands in `benchmarks/results_suite.csv` and selected configurations in `benchmarks/tuned_configs.json`; both are committed as the provenance for every table in Section 6. Datasets download via scikit-learn; Friedman1 is synthetic with a fixed seed. Figures reproduce exactly only on the same C++ standard library (Section 3.8).

The compression and serialized-size measurements in Sections 3.6 and 3.9 were taken at the shipped defaults with:

```
m = SXERegressor(random_state=0).fit(X, y)
nodes, leaves = m.ensemble_.structure(m._members)
blob     = len(m.ensemble_.serialize())   # == m.tree_bytes_
portable = (nodes - leaves) * 13 + leaves * 8
```

The prototype benchmarks in `reference/` still run from a checkout, and are the source of the v1/v2 figures in Section 6:

```
cd reference
g++ -O3 -march=native -fopenmp -shared -fPIC sxe.cpp -o libsxe.so
python benchmark.py        # v1: RandomForest + LightGBM baselines + Python SXE
python cpp_benchmark.py    # v2: multi-round C++ SXE
python parity_check.py     # the packaged core against the prototype
```

12 Conclusion

SXE is a coherent point in the design space between the random forest and the gradient-boosted machine, reached by asking what happens when a boosting step is a small forest rather than a single tree. The three devices built on that — randomized members, per-member holdout pruning at randomized strength, and a non-negative lasso that deletes most of the trajectory — are all variance-reduction devices, and the empirical results track that fact closely: the method wins where variance dominates and loses where bias does, on a head-to-head against a tuned LightGBM that splits four–five on accuracy and five–four on size.

The engineering claims are stronger than the accuracy claim and are the more durable part of the contribution. Reproducibility across thread counts is structural rather than best-effort, because per-tree seeds derive from tree indices and both growers recurse in the same order; it is tested, not asserted. The serialized artefact is now smaller than the portable formula used to compare libraries, so the advertised size is conservative. And the parallel structure — independent trees within a batch, one serial update between batches — is a genuine architectural advantage over sequential GBDT that also maps unusually cleanly onto a GPU.

The honest headline is the negative one. The shipped defaults give up a median 9.5% error reduction to tuning where LightGBM's give up 3.3%, and the tuned configurations agree on the direction of the fix: deeper trees, more batches, larger batches. The method is competitive; the product is not yet. Closing that specific gap is worth more than any of the accuracy refinements on the list, because a user's first fit is what they actually get.

References

[1] A. Agarwal, Y. S. Tan, O. Ronen, C. Singh, and B. Yu. Hierarchical shrinkage: Improving the accuracy and interpretability of tree-based methods. In *ICML*, 2022.

[2] L. Breiman. Out-of-bag estimation. Technical report, Statistics Department, UC Berkeley, 1996.

[3] L. Breiman. Stacked regressions. *Machine Learning*, 24(1):49–64, 1996.

[4] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[5] L. Breiman, J. Friedman, R. Olshen, and C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[6] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *KDD*, 2016.

[7] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani. Least angle regression. *Annals of Statistics*, 32(2):407–499, 2004.

[8] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[9] J. H. Friedman and B. E. Popescu. Importance sampled learning ensembles. Technical report, Stanford University, 2003.

[10] J. H. Friedman and B. E. Popescu. Predictive learning via rule ensembles. *Annals of Applied Statistics*, 2(3):916–954, 2008.

[11] P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.

[12] L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *NeurIPS Datasets and Benchmarks*, 2022.

[13] T. Hastie, J. Taylor, R. Tibshirani, and G. Walther. Forward stagewise regression and the monotone lasso. *Electronic Journal of Statistics*, 1:1–29, 2007.

[14] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS*, 2017.

[15] J. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.

[16] J. R. Quinlan. Simplifying decision trees. *International Journal of Man-Machine Studies*, 27(3):221–234, 1987.

[17] S. Wager and S. Athey. Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, 113(523):1228–1242, 2018.

A Parameter reference

Shipped defaults. Parameters follow scikit-learn's `HistGradientBoosting*` vocabulary where the concepts match; the third column gives the name used by the prototype in `reference/`, for reading that code. The five values marked † were revised after the benchmark run (Section 10.3); the results of Section 6 were measured at the previous settings, shown in brackets.

Parameter	Default	Prototype	Meaning
Boosting trajectory			
`learning_rate`	`0.07`† [0.15]	`shrink`	ν, shrinkage on each batch-average correction
`max_iter`	`300`† [120]	`max_batches`	maximum boosting batches
`batch_size`	`16`† [10]	`batch`	trees grown in parallel per batch
`n_iter_no_change`	`15`† [8]	`patience`	early-stopping patience, in batches
`tol`	`1e-4`	—	improvement threshold, in units of $\mathrm{Var}(y_{\mathrm{val}})$
`validation_fraction`	`0.18`	`val_fraction`	fold for early stopping, weights and calibration
Per-tree randomization			
`max_depth_choices`	`(6,8,10,12)`† [(4,5,6,7,8)]	`depth_choices`	depth sampled uniformly; `0` ⇒ 64
`min_samples_leaf_range`	`(10, 60)`	`leaf_range`	log-uniform per tree
`max_features_range`	`(0.5, 1.0)`	`feat_range`	candidate-feature fraction, uniform per tree
`subsample_range`	`(0.4, 0.75)`	—	row subsample; complement is the pruning holdout
`prune_strength_range`	`(1e-3, 1.0)`	`lam_range`	λ, log-uniform per tree
Tree construction			
`splitter`	`"best"`	`argmax_bins`	exact bin scan, or `"random"` cut-points
`n_thresholds`	`2`	`n_thresh`	random cut-points when `splitter="random"`
`max_leaf_nodes`	`None`	`max_leaves`	if set, leaf-wise growth with this budget
`max_bins`	`255`	`n_bins`	bins for finite values; a 256th holds missing
`bundle_binary_features`	`False`	`bundle_binary`	exclusive feature bundling for binary columns
`hybrid_min`	`600`	—	node size above which subtraction is used (speed only)
Weight fit and output			
`l1_ratio`	`1.0`	`l1_ratios`	`1.0` is a pure lasso
`n_alphas`	`20`	`n_alphas`	regularisation path length
`weight_cv`	`3`	`cv`	folds for choosing α
`calibrate`	`"sigmoid"`	—	Platt scaling; `None` clips to $[10^{-6}, 1-10^{-6}]$
Execution			
`n_jobs`	`None`	—	threads; `None` ⇒ all cores. Does not affect the model
`random_state`	`None`	—	seeds binning, the split, and per-tree randomization

B Tuning guidance

Distilled from the nine per-dataset searches. Across all of them the same three moves recurred — **deeper trees, more batches, a larger batch size** — and no winning configuration used the default depth choices.

Smooth, low-dimensional signal.
Fine trajectories: small `learning_rate`, many batches, small `batch_size`, mild subsampling (0.85–1.0), `splitter="best"`, `max_leaf_nodes` ≈ 63.

High-dimensional or one-hot data.
`max_depth_choices = (10, 12, 14)`, `splitter = "random"` with `n_thresholds = 3`, `bundle_binary_features = True`, moderate subsampling.

Noisy data with irrelevant features.
Aggressive subsampling (0.2–0.4) and a large `learning_rate` with tight `n_iter_no_change`. This is also where the speed comes from.

C Diagnostics exposed by a fitted model

```
model.summary()             # batches, trees grown/kept, leaves, size, timings
model.feature_importances_  # split-gain, weighted by ensemble weight
model.n_iter_               # batches actually run before early stopping
model.n_trees_, model.n_trees_kept_
model.model_size_bytes_     # the bytes serialize() writes, not an estimate
```

With `bundle_binary_features=True`, a bundle's gain is divided equally among its member columns — a threshold on a bundle separates a group of them, not one — so those entries are a per-bundle figure spread over its columns rather than a per-column measurement.